LLM-Guided Grammar-Aware Fuzzing of PostgreSQL

CS-5187 Term Project A3 — Progress meeting 2: results

Venu Bhuvan Gowtham

26–30 October 2026

Status at a glance

Done since meeting 1

- Every item of the meeting-1 plan implemented and tested on real PostgreSQL master
- 4 previously unknown PostgreSQL bugs: 2 confirmed by committers, 1 fixed on master
- 12 metamorphic oracle kinds on derivation trees, 18 plan variants, cross-version
- Triage: tree + token reduction, localization, upstream drafts
- Gated LLM arm; 23-bug re-injected benchmark
- Campaigns finished (RQ1–RQ4); paper at 12 pages with real numbers

Numbers

- Grammar: 36.7% of lines vs 22.2% random; 99% valid vs 26%
- Oracles double plan shapes (20k $\rightarrow$ 34k)
- 93 candidates triaged: 7 real, 15 duplicates, precision 0.16 (0.37 with duplicates)
- Re-injected bugs: at most 2 of 20 in 1 h
- LLM: +5 features, no coverage gain
- 148 unit tests, $\approx$10.5k lines of Python

Next: paper, final slides, verify open candidates

Recap: problem and the three claims

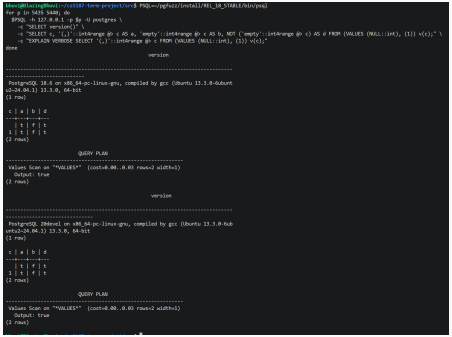
Problem. PostgreSQL's planner changes every week. The worst bugs are silent wrong results. Crash fuzzers miss them; SQLancer-style oracles use small hand-written generators; neither knows what changed in yesterday's commit.

1. **One tree, four jobs.** A typed derivation tree generates, mutates, is rewritten by the oracles, and is reduced in triage. **all in code**
2. **Soundness we measure.** Generator validity; per-oracle precision before/after each false-positive class is fixed. **ledger**
3. **The LLM proposes, the server decides.** Fragments and rewrites enter only through a gate on the real server. **gate**

Ladder: Random/Mutation → Grammar → Grammar+Oracles → +LLM, same server, oracles, budget, seeds.

Motivating example: BUG-02 (found by our oracle)

```
SELECT c,
  '(,)'::int4range @> c       AS a, -- want NULL
  'empty'::int4range @> c     AS b, -- want NULL
  NOT ('empty'::int4range @> c) AS d
FROM (VALUES (NULL::int), (1)) v(c);
-- 17, 18.6, master: NULL row ->  t | f | t
```



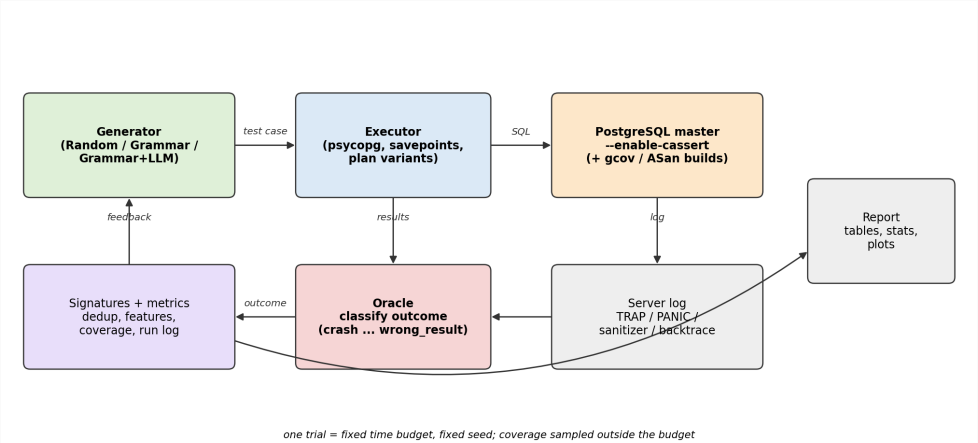
- Strict operator, but a planner *support function* folds it to `true`/`false` (`rangetypes.c`)
- Silent wrong result in released PostgreSQL 17 and 18
- Found because the oracle knew `'(,)'::int4range` is one typed `expr<int4range>` node: e ≡ `(SELECT e)`

What changed since meeting 1

Plan item	What we built	Status
Metamorphic oracles	TLP (WHERE, HAVING, aggregates), NoREC, expression, predicate, NOT NULL, ORDER BY, CTE, fence	done
Cross-version	master vs 18.6, per-worker DBs, intended-change list	done
Triage	reproduce $\times 3$, tree + token reduction, localization, upstream draft, bisect	done
LLM loop	miner, cached prompt, gate, diff coverage, repair, ledger, rewrite gate	done
Benchmark (RQ3)	23 reverted fixes; fixed / solo / injected servers; attribution	done
Stretch goals	plan-shape guidance, bandit weights, data profile, RLS and CERT oracles	done
Campaigns	RQ1–RQ4: 5 trials $\times$ 1 h per arm, two machines; LLM arm with Gemini Flash (free tier)	done
Upstream	4 bugs filed or confirmed (#19768, #19793, #19802, #19803)	done

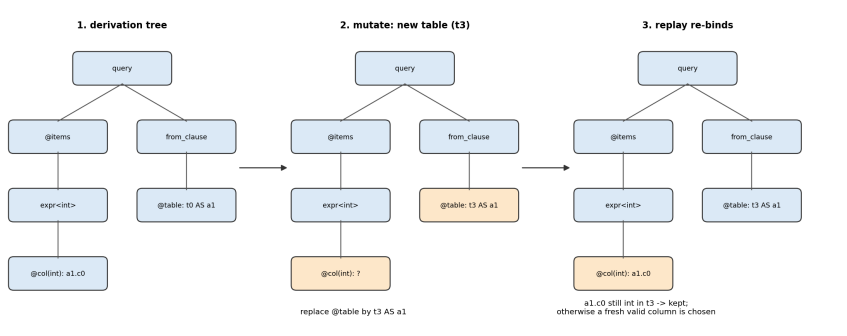
The system

Architecture



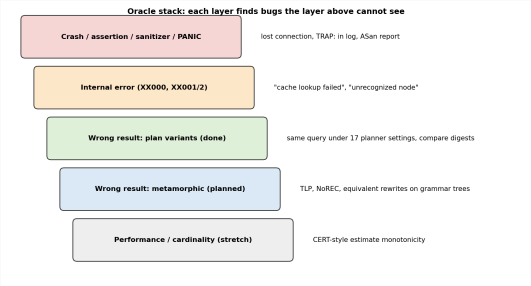
Fuzzing loop (top): derive → attach checks → run several ways → classify/triage. LLM loop (bottom): commit → proposal → gate → grammar.

One tree, four jobs



Generate	typed grammar: 78 rules, 421 alternatives, scopes + type variables
Mutate	regen / splice / insert / drop subtrees; *replay* re-binds names and types
Oracle	rewrite one typed node or clause span $\rightarrow$ equivalent query
Reduce	replace subtrees by their smallest derivation; replay keeps SQL valid

The oracle stack



- **Crash / assertion / internal error** on every statement (cassert build, log + PID check)
- **18 plan variants**: join methods, hash agg, parallel, partition-wise, tiny `work_mem`, self-join elim., eager agg, JIT, ...
- **12 metamorphic kinds** on the tree
- **Cross-version**: master vs 18.6
- Determinism filter, canonical digests (numeric scale, −0, float classes), confirmation re-run

Metamorphic rewrites on derivation trees

TLP-WHERE (generated + rewritten by the tool)

```
SELECT t0_1.c0 FROM t0 AS t0_1
  ==
(SELECT .. WHERE (t0_1.c0 > 5))
UNION ALL (SELECT .. WHERE NOT (t0_1.c0 > 5))
UNION ALL (SELECT .. WHERE (t0_1.c0 > 5) IS
    NULL)
```

Also: TLP-HAVING, TLP-aggregates (MIN of MINs, SUM of COUNTs), NoREC, `IN`↔`OR`, a<b↔`NOT`(a>=b), `BETWEEN`, NOT NULL conjunct, drop `ORDER BY`, CTE (not) materialized, `OFFSET 0` fence, typed expression identities.

Why the tree matters

The tree knows *which* `WHERE` belongs to which `SELECT`, each node's type, grouped levels, scopes and outer joins. So each rewrite is applied only where it is sound:

- multiset contexts only
- `DISTINCT`: unique-text types only
- NOT NULL: never under outer joins
- `e+0` on `int`, not on `float8`

Beyond wrong results: RLS leak oracle and CERT

RLS leak oracle (information flow, L21–22)

- Policies on 5 tables, role `pgfuzz_rls` (sees 131 of 203 rows of `t0`)
- Left: query as the restricted role
- Right: query as superuser with each table replaced by `(SELECT * FROM t WHERE policy)`
- More rows on the left = leak
- 532 checks, 0 false positives

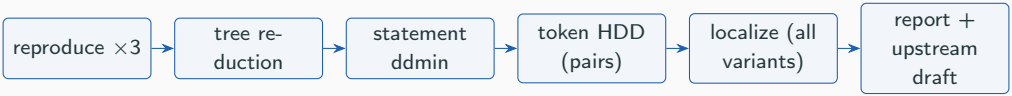
CERT (Ba & Rigger, ICSE'24)

- Adding `WHERE` must not *raise* the planner's row estimate
- Outcome `performance`, reported apart from wrong results
- 58 checks, 0 violations

Cross-version

1,155 comparisons, 0 mismatches after fixing clock data and excluding intended changes (`corr` on master)

Triage: from failure to upstream report



BUG-01 case (590 tokens)	tokens	runs
token reduction only	116	406
tree first, then tokens	**79**	555

- Localization: BUG-02 → “not toggled by planner settings” → support function (correct)

```
bhuvi@blazingBhuvi:~/cs5187-term-project/src$ cat ~/cs5187-term-project/runs/memberA-rq3/logs/A3_triage.txt
[cde0712bc8db] assertion via assertion: reproducing x3
  statements: 3 -> 2
  statements: 2 -> 1
  clauses: 572 -> 570 tokens
  clauses: 570 -> 543 tokens
  clauses: 543 -> 529 tokens
  clauses: 529 -> 527 tokens
  clauses: 527 -> 515 tokens
  items: 515 -> 420 tokens
  items: 420 -> 410 tokens
  items: 410 -> 318 tokens
  items: 318 -> 314 tokens
  items: 314 -> 290 tokens
  items: 290 -> 277 tokens
  items: 277 -> 273 tokens
  items: 273 -> 269 tokens
  items: 269 -> 265 tokens
  items: 265 -> 263 tokens
  items: 263 -> 261 tokens
  groups: 261 -> 201 tokens
  groups: 201 -> 162 tokens
  groups: 162 -> 141 tokens
  groups: 141 -> 139 tokens
  groups: 139 -> 125 tokens
  groups: 125 -> 116 tokens
  reduced to 1 statement(s), 116 tokens in the culprit (406 runs, 73.3 s)
1 candidate(s) triaged into ../results/triage_demo; verdict template: ../results/triage_demo/verdicts.template.yaml
bhuvi@blazingBhuvi:~/cs5187-term-project/src$
```

The LLM arm: the LLM proposes, the server decides

1. Mine master commits (planner, executor, parser), excluding the 23 benchmark fixes
2. Cached 9k-token prompt + commit digest → fragment, seeds, rewrites (JSON schema)
3. **Gate**: parse · derivable ($\geq$5 uses) · valid on server ($\geq$60%) · seeds run · diff coverage up
4. Repair rounds with the server's errors (max 3); ledger of tokens and cost

API or manual mode; stored responses replay for free.

Rewrite gate (value probe + 150 live test cases)

Proposed rewrite	Result
`CASE WHEN e IS NULL THEN NULL ELSE e END`	accepted, 90/90
`substr(e, 1)`	accepted, 88/88
`e + 1`	rejected: 12 mismatches
`e IS TRUE`	rejected: probe at NULL

`e IS TRUE` passed 109 sampled checks: sampling is not proof.

Evaluation

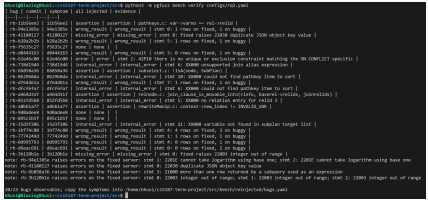
RQ3 benchmark: 23 real bugs put back into PostgreSQL

- Fixes from 31 Jul–16 Oct 2026 with regression tests, reverted on base `26bd08fb`
- Servers: **fixed**, **solo**$_i$ (one bug), **injected** (all 23)
- `bench verify`: symptom = fixed vs solo difference
- `bench attribute`: replay finding on each solo server

Key fact

20/23 observable, only 9 visible to a crash-only fuzzer.

Symptom	Bugs
wrong result	8
internal error	5
assertion	4
missing / wrong error	3
not observable (excluded)	3



Component checks on master (single runs)

Component	Measured
Metamorphic oracles (first run)	907 checks, 0 FP; BUG-02 in the first 150 test cases
New rewrite kinds	798 checks, 0 false positives
Full system, 3 min	3,166 test cases, 99.5% valid, 1,991 checks, 7,956 variant runs, 0 FP
Bandit weights	533 vs 419 plan shapes per 1,000 test cases, same validity
Data profile (`@dlit`)	selective predicates (5–95% of rows): 29.3% $\rightarrow$ 58.6%
Tree replay	599/600 stored trees replay to identical SQL
Token reduction (BUG-02)	4 statements / 107 tokens $\rightarrow$ 1 / 36 in 307 runs
LLM fragment gate	valid fragment 251/251 accepted; invalid rejected with server message

Precision ledger (RQ2): 93 triaged candidates

Oracle	Cand.	Real	Dup.	FP	Flaky	Open
assertion	15	3	10	1	1	0
cross-version	25	0	0	7	18	0
internal error	4	2	1	0	1	0
mm: expr	3	1	1	1	0	0
mm: rls	5	0	0	0	0	5
mm: tlp_having	4	0	0	4	0	0
mm: tlp_where	4	1	1	2	0	0
plan-variant	32	0	2	21	8	1
all (+1 crash)	93	7	15	37	28	6

Precision $7/44 = 0.16$; $22/59 = 0.37$ if duplicates count. Not re-measured after the fixes.

10 false-positive classes, all fixed + tested

- 6 before the campaigns (`ARRAY(SELECT)`, TLP-HAVING, numeric scale, seed clock, PoC PID, intended `corr` change)
- Equal intervals with different text: 16
- `ORDER BY NULL` in ordered aggregates: 2
- cross-version state divergence (`IGNORE NULLS`): 5
- LLM rewrite changing the collation: 1

Oracle precision, not generator power, is the bottleneck.

Bugs found

BUG-01 — JSON_TABLE collations

- Found by the grammar campaign (assertion, `execExpr.c:2078`)
- 18.6: `ROW(1,2) < ROW(3,4)` in the context item returns false
- Missing `assign_expr_collations()` in `parse_jsontable.c`

BUG-02 — range containment w/ NULL

- Found by `expr:scalar_sub` rewrite
- `find_simplified_clause()` folds to a constant

Bug	Found by	Upstream	Status
BUG-01 JSON_TABLE collations	grammar	#19768	filed
BUG-02 range containment NULL	`(SELECT e)` rewrite	#19793	filed, patch
MERGE "SubPlan with no parent"	grammar	#19802	confirmed, patch posted
Parallel Append assertion	LLM fragments + TLP	#19803	confirmed, fixed on master

Rediscovered after others: #19800 (self-join elimination, plan-variant oracle), #19772, #19784, #19771.

Experiment design (RQ1–RQ4)

RQ1 reach: validity, line/branch coverage, features, plan shapes

RQ2 bugs found and per-oracle precision

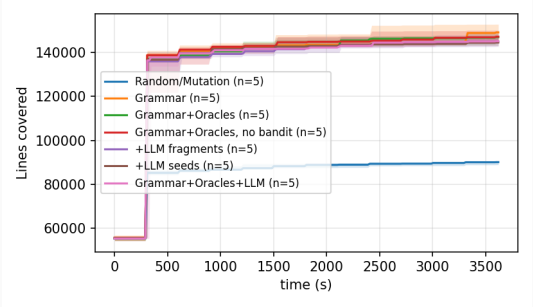
RQ3 re-injected bugs found, time to find

RQ4 what the LLM adds (diff coverage, ablation) and costs

- RQ1: 7 arms (incl. no-bandit and LLM ablations) $\times$ 5 trials $\times$ 1 h, gcov build
- RQ3: 4 arms $\times$ 5 $\times$ 1 h, injected server + 18.6 reference
- Paired seeds, same oracles and budget; one commit (`26bd08fb`) for everything
- Medians, IQR, bootstrap 95% CI, Mann–Whitney U, Vargha–Delaney $\hat{A}_{12}$

RQ1: reach (median of 5 one-hour trials)

Arm	Valid	Lines	Branches	Features	Plan shapes	Tests
Random	26.0%	22.2%	15.1%	23	43	1,074k
Grammar	99.3%	36.7%	27.1%	101	20,279	71k
Grammar+Oracles	96.3%	36.2%	26.7%	101	34,313	59k
no bandit	99.4%	36.2%	26.8%	101	23,997	65k
+LLM fragments	95.5%	35.7%	26.5%	104	25,972	48k
+LLM seeds	98.3%	35.6%	26.4%	106	29,274	64k
+LLM both	95.3%	35.7%	26.4%	106	26,375	47k



- Grammar vs random: $\hat{A}_{12} = 1.00$, $p = 0.008$, though random runs 15× more tests
- Oracles: no line gain ($p = 0.42$), plan shapes 20k → 34k ($\hat{A}_{12} = 0.84$, $p = 0.095$)
- Oracles cost throughput: 67 → 40 statements/s

RQ3 and RQ4

RQ3: bugs found (of 20), 5 trials $\times$ 1 h

	Rand.	Gram.	+Orac.	+LLM
`dfcf6fa7` (error)	0/5	5/5	4/5	5/5
`11b56ee2` (assert.)	0/5	0/5	1/5	1/5
wrong result (8)	0	0	0	0
bugs found	0	1	2	2

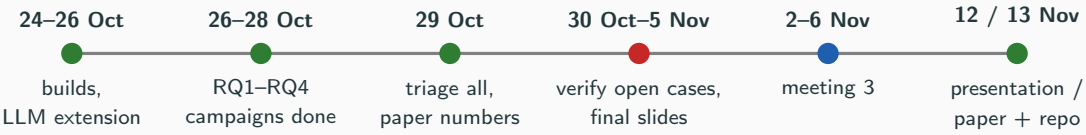
- The benchmark does *not* show an oracle advantage; that rests on RQ2
- 1 h $\times$ 5 trials is short for bugs needing feature combinations

RQ4: LLM (Gemini Flash, free tier)

- 10 commits, 4 accepted (40%), 25 rounds; 16 rejected on server validity
- +5 SQL features ($p = 0.004$), no coverage gain
- 328k in / 171k out tokens, 0 USD
- 7/7 rewrites passed the gate; one changed the collation $\rightarrow$ false positive; gate now rejects `COLLATE`
- Fragments-only arm found #19803

Plan

Timeline to the deadline



- Open: 5 RLS candidates and one `corr()` case to verify; authorship of fix 776df8a6
- Results and deviations: `docs/RESULTS_ANALYSIS.md`

Risks and what we would like feedback on

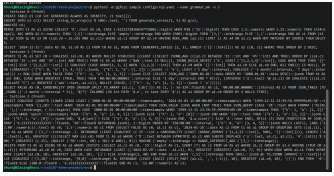
Risks and fallbacks

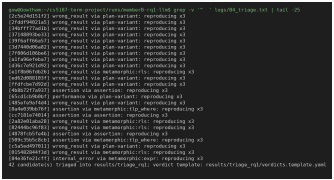
- Happened: 4 new false-positive classes; fixed after the runs, not re-measured
- Happened: no paid key, LLM arm ran on Gemini Flash, 10 commits (quota)
- 5 trials rarely reach $p < 0.05$ $\rightarrow$ we report effect sizes
- Remaining upstream replies after 13 Nov $\rightarrow$ report status as of the deadline

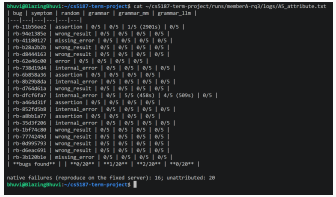
Questions for you

- Do two committer-confirmed bugs (one fixed) meet the top band even though the benchmark shows no oracle advantage?
- Is it acceptable that precision after the fixes is argued from reproducers, not a new campaign?

Live demo (backup screenshots)







Thank you — questions?

github.com/v28k/cs5187-term-project